

SEMBOT CSS · DOKUMENTACJA TECHNICZNA

CSS Onboarding

Pełna ścieżka użytkownika

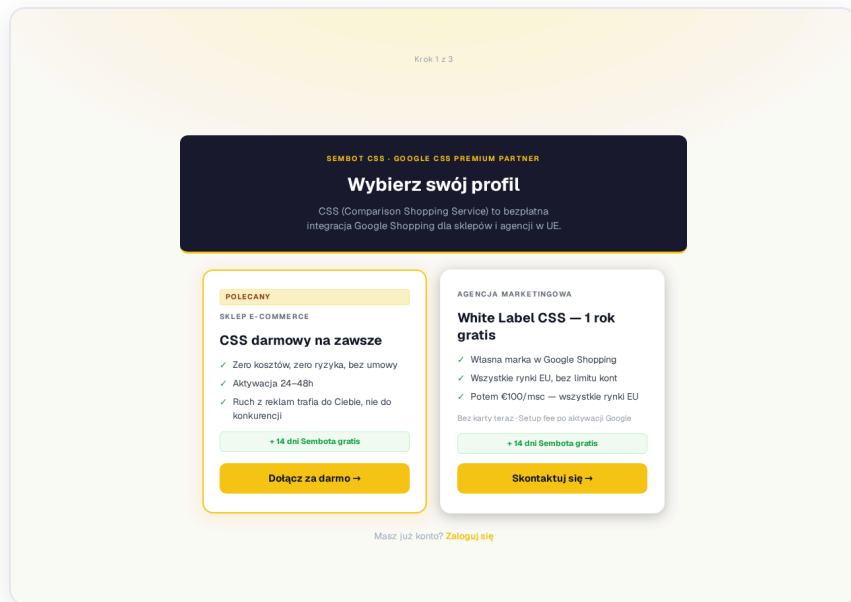
Dokumentacja wszystkich ekranów, logiki frontend i backend dla ścieżki CSS Onboardingu w nowym Sembocie (proxy_sembot + sembot-laravel).

Opisuje: wybór profilu, rejestrację, formularz sklepu, ścieżkę agencji, stronę sukcesu oraz mechanizm trial i auto-downgrade.

Wygenerowano: 25 czerwca 2026 · Serwis: [app.sembot.com](#) · CSS: [css.sembot.com](#)

Spis treści

1. Krok 1/3 — Wybór profilu CSS	str. 3
2. Krok 2/3 — Rejestracja (ścieżka Sklep)	str. 4
3. Krok 2/3 — Rejestracja (ścieżka Agencja)	str. 5
4. Krok 3/3 — Formularz CSS Shop	str. 6
5. Krok 3/3 — CSS Agency (przed wysłaniem maila)	str. 7
6. CSS Agency — po kliknięciu przycisku email	str. 8
7. Strona sukcesu — CSS Shop aktywowany	str. 9
8. Mechanizm Trial i Auto-Downgrade	str. 10



Punkt wejścia dla ruchu przychodzącego z **css.sembot.com**. Użytkownik widzi dwie karty:

- **Sklep e-commerce** — wyróżniony jako "Polecany" (żółty border, badge). CSS bezpłatny na zawsze + 14 dni Sembota gratis.
- **Agencja marketingowa** — White Label CSS, 1 rok gratis, potem €100/msc.

Wskaźnik kroków *Krok 1 z 3* orientuje użytkownika w procesie.

FRONTEND

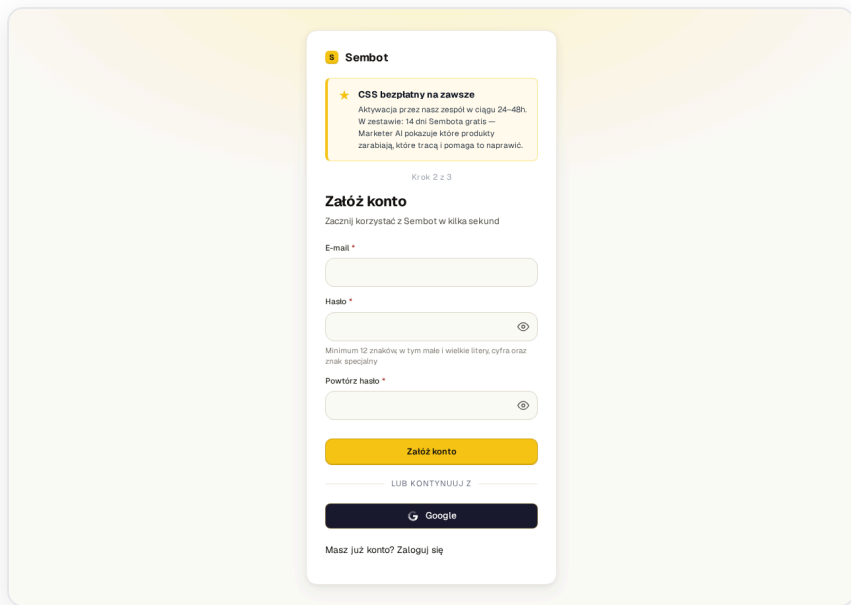
- Templ: `CSSPath()` w `css_path.templ`, montowany w `AuthLayout`.
- Ciemne tło (`#1a1a2e`) dopasowane do klimatu `css.sembot.com`.
- Hover na kartach: żółty border + cień. Karta sklepu zawsze podświetlona.
- CTA sklep: *Dołącz za darmo* → | CTA agencja: *Skontaktuj się* →
- Brak Alpine.js — statyczny HTML, zero JS na tym ekranie.

BACKEND (GO)

- Route: `GET /auth/css` → handler `ShowCSSPath`.
- Strona publiczna — brak auth guard.
- UI context: locale (PL/EN), theme, whitelabel branding z cache.

BACKEND (LARAVEL)

Brak wywołań API. Ekran jest w pełni statyczny po stronie serwera Go.



Standardowy formularz rejestracji wzbogacony o kontekst CSS. Nad formularzem pojawia się banner CSS z żółtym obramowaniem.

FRONTEND

- Templ: `Signup(SignupVM{CSSType: "css-shop"})` .
- Banner: ikona ★, tytuł "CSS bezpłatny na zawsze", opis z Marketer AI.
- Wskaźnik kroków: *Krok 2 z 3*.
- Ukryte pole `css_type=css-shop` — przekazuje kontekst do handlera.
- Walidacja hasła Alpine.js: min. 12 znaków, wielka litera, mała litera, cyfra, znak specjalny. Komunikat inline pod polem. Submit zablokowany gdy hasło niepoprawne.
- HTMX: błędy serwera (np. email zajęty) wstrzykiwane inline bez przeładowania.

BACKEND (GO)

- `GET /auth/register` → `ShowSignup` czyta `?type=` z query, ustawia `SignupVM.CSSType` .
- `POST /auth/register` → `DoSignup` : walidacja, wywołanie `RegisterUser` API, zapis JWT do sesji AES-GCM, redirect do `/css/shop` (dla `css-shop`) lub `/css/agency` .

BACKEND (LARAVEL)

- `POST /api/v1/auth/register` : tworzy usera + workspace.
- `workspace.trial_ends_at = now() + 14 dni` ustawiane automatycznie w `UserRepository::createAccount` (gdy `trialPaymentRequired=false`).
- Zwraca JWT Bearer token + workspace slug.

Sembot

★ **White Label CSS — 1 rok bezpłatnie**
Własna marka w Google Shopping. W zestawie: 14 dni Semбота gratis — Marketer AI zarządza kampaniami wszystkich klientów z jednego miejsca.

Krok 2 z 3

Załóż konto
Zaczniij korzystać z Sembot w kilka sekund

E-mail *

Hasło *

Minimum 12 znaków, w tym małe i wielkie litery, cyfra oraz znak specjalny

Powtórz hasło *

Załóż konto

LUB KONTYNUUJ Z

Google

Masz już konto? Zaloguj się

Identyczny formularz rejestracji, ale banner CSS pokazuje treść dla agencji.

FRONTEND

- Banner: "White Label CSS — 1 rok bezpłatnie" + opis Marketer AI dla agencji.
- Ukryte pole `css_type=css-agency`.
- Walidacja hasła i reszta formularza — identyczna jak ścieżka sklepu.

BACKEND (GO)

- Po rejestracji: redirect do `/css/agency` zamiast `/css/shop`.

BACKEND (LARAVEL)

Identycznie jak ścieżka sklepu — trial 14 dni aktywowany przy rejestracji.

Krok 3 z 3

SEMBOOT CSS · GOOGLE CSS PREMIUM PARTNER

Twój CSS jest bezpłatny na zawsze

Uzupełnij dane — nasz zespół aktywuje Twój CSS w ciągu 24-48h.

★ **W zestawie: 14 dni Sembota gratis**
Widzisz które produkty zarabiasz, które tracą i dlaczego. Marketer AI pomaga to naprawić — wystarczy że powiesz co chcesz osiągnąć.

Nazwa sklepu *

np. Mój Sklep

URL pliku produktowego feeda *

https://

URL pliku produktowego z Twojego sklepu, Shopify: Kanady sprzedaży → Google → URL feeda. WooCommerce: wtyczka Product Feed. PrestaShop: Moduły → Google Merchant Center.

Domena sklepu *

np. mojsklep.pl

GMC ID (Google Merchant Center) (opcjonalne)

np. 123456789

Gdzie znaleźć GMC ID? Zaloguj się na [merchants.google.com](#) → liczba w lewym górnym rogu lub w URL (7a=000000000). Nie masz konta? [Utwórz konto](#) — zajmuje ok. 10 minut.

Aktywuj CSS i przejdź do panelu

Twój projekt w Sembocie zostanie stworzony automatycznie

Formularz zbierający dane potrzebne do aktywacji CSS. Pełni funkcję onboardingu — po wypełnieniu projekt w Sembocie jest gotowy automatycznie.

FRONTEND

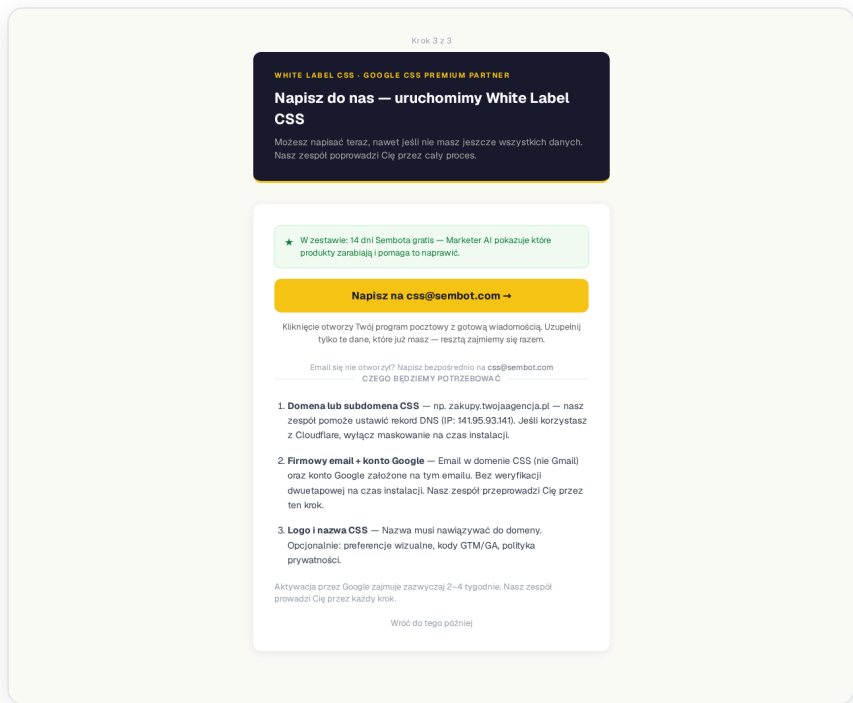
- Templ: `CSSShopOnboarding(vm)`. Layout `layouts.Base` bez sidebar/brandbar — fullscreen onboarding.
- Trial box: ★ zielony, "W zestawie: 14 dni Sembota gratis" + opis Marketer AI.
- Kolejność pól: **Nazwa sklepu** (wymagane) → **URL feeda** (wymagane) → **Domena** (auto-fill) → **GMC ID** (opcjonalne).
- Auto-fill Alpine.js: wpisanie feed URL automatycznie ekstrahuje hostname i wypełnia pole Domena (`new URL(feedUrl).hostname.replace(/^www\./, '')`).
- Submit: loading state "Tworzę projekt..." + disabled button (Alpine.js `submitting` flag).
- Hint GMC ID: link do [merchants.google.com](#) z instrukcją gdzie znaleźć ID.

BACKEND (GO) — SUBMITSHOP

1. `CreateProject(ProjectTypeSimple, domain, name)` → tworzy projekt w Laravel jako **Simple (type=2)**.
2. `CreateInputFeed(DefaultCSSFeedMapping())` → 18-polowe mapowanie Google Shopping XML (id, title, description, link, image_link, price, brand, gtin, condition, availability...).
3. `CSSShopNotify({ShopName, GmcID, FeedURL, Domain, ProjectURL, ProjectID})` → powiadomienie do CS team + ustawienie `in_css_program=true`.
4. Redirect do `/css/shop/done?ws=&proj=&name=`.

BACKEND (LARAVEL)

- `POST /api/v1/projects` : projekt tworzony synchronicznie (`dispatch_sync`), zwraca ID+slug natychmiast.
- `POST /api/v1/feeds/input/{id}` : feed z domyślnym mapowaniem CSS → import produktów startuje w ciągu minut.
- `POST /api/v1/css/shop/notify` : mail do `css@sembot.com` , ustawia `project.in_css_program = true` (flaguje projekt do downgrade po trialu).



Ekran dla agencji po rejestracji. Zamiast formularza — gotowy szablon email do CS team.

FRONTEND

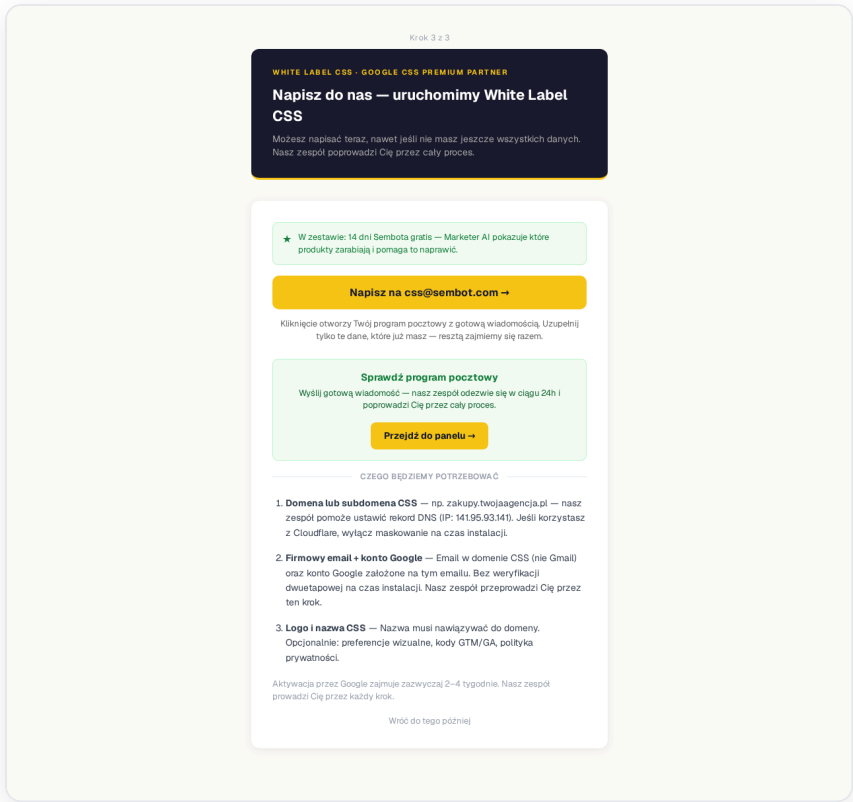
- Templ: `CSSAgencyOnboarding()` .
- Trial reminder (zielony box): przypomnienie o 14 dniach Sembota gratis z opisem Marketer AI.
- Przycisk "Napisz na `css@sembot.com` →": otwiera klienta email z pre-wypełnionym tematem i treścią (`mailtoCSS(locale)` — PL lub EN zależnie od locale usera).
- Alpine.js state: `emailClicked: false` opakowuje formularz. Kliknięcie przycisku ustawia flagę → zmienia widok.
- Fallback (gdy email nie otworzył się): "Email się nie otworzył? Napisz bezpośrednio na `css@sembot.com`"
- Lista wymagań: domena/subdomena CSS, email firmowy + konto Google, logo i nazwa CSS.
- Timeline: aktywacja 2–4 tygodnie od przesłania danych.

BACKEND (GO)

- Handler: `ShowAgency` — wymaga sesji, renderuje stronę. Zero wywołań API.
- `mailtoCSS(locale)` : czysta funkcja Go generująca URL mailto z pre-encoded treścią po polsku lub angielsku.

BACKEND (LARAVEL)

Brak. Agencja kontaktuje się bezpośrednio emailem — dalszy setup White Label CSS prowadzony ręcznie przez CS team.



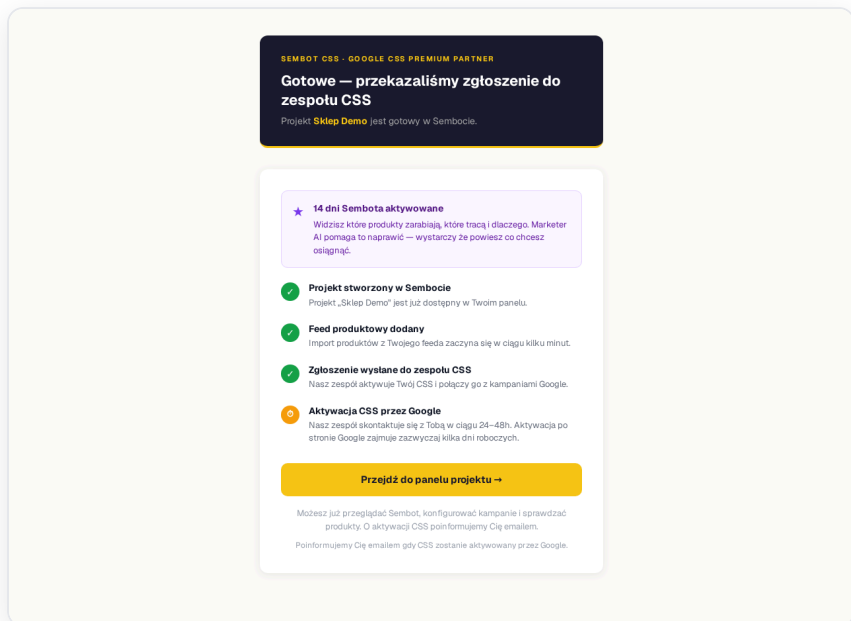
Stan ekranu po kliknięciu "Napisz na `css@sembot.com` →". Alpine.js przełącza widok — użytkownik widzi potwierdzenie i CTA do panelu.

FRONTEND — ZMIANA STANU

- `emailClicked` przełącza się na `true` przez `x-on:click` na przycisku mailto.
- Zielony box potwierdzenia: "Gotowe — email otwarty. Wyślij wiadomość, a nasz zespół odezwie się w ciągu 24h."
- Pojawia się przycisk "Przejdź do panelu →" prowadzący do dashboardu Sembota.
- Znika fallback "Email się nie otworzył?" (był widoczny gdy `emailClicked=false`).
- Całość: `x-show="emailClicked"` / `x-show="!emailClicked"` — zero JS poza Alpine.js, zero dodatkowych request do serwera.

BACKEND

Brak. Zmiana stanu jest wyłącznie client-side — Alpine.js reactive data bez żadnego HTTP request.



FRONTEND

- Templ: `CSSShopSuccess(vm)` .
- Hero z nazwą sklepu w tytule (żółty kolor).
- **Bonus card** (fioletowy, ★): "14 dni Sembota aktywowane" z opisem Marketer AI — wydzielona karta ponad checklistą, nie element chronologicznej listy.
- Checklist statusów:
 - ✓ Projekt stworzony w Sembocie — zielony
 - ✓ Feed produktowy dodany — zielony
 - ✓ Zgłoszenie wysłane do zespołu CSS — zielony
 - ⌚ Aktywacja CSS przez Google — amber (oczekuje)
- CTA "Przejdź do panelu projektu →" → `/p/{ws}/{proj}/feed` (widok feedu, gdzie już trwa import produktów).
- Notatka: "Poinformujemy Cię emailem gdy CSS zostanie aktywowany przez Google."

BACKEND (GO)

- Handler: `ShowShopDone` — wymaga sesji, czyta `ws` , `proj` , `name` z query string.
- Fallback: gdy brak `ws` w URL — używa `sess.WorkspaceSlug` .
- Zero wywołań API — dane z sesji + query params.

BACKEND (LARAVEL)

Brak wywołań na tym ekranie. Wszystko co wymagało API zostało wywołane przez `SubmitShop` przed redirectem.

Trial — jak działa

Przy rejestracji (`UserRepository::createAccount`) Laravel automatycznie ustawia `workspace.trial_ends_at = now() + 14 dni` gdy `trialPaymentRequired = false` . Trial jest na poziomie **workspace**, nie projektu — obejmuje wszystkie funkcje Sembota (Marketer AI, monitor pozycji, optymalizator feedów, raporty kampanii).

Projekt jako Simple z trialem

Projekt CSS tworzony jest jako `ProjectType::SIMPLE` (2) . Typ Simple blokuje zaawansowane funkcje przez `BlockInSimpleProjects` middleware i `QuotaValidator::projectTypeAllows()` . W czasie trwania trialu workspace ma pełen dostęp niezależnie od typu projektu — po jego zakończeniu Simple blokuje kampanie, monitoring i output feedy.

Oznaczenie projektu CSS — `in_css_program`

Endpoint `POST /api/v1/css/shop/notify` (wywoływany przez Go po stworzeniu projektu) ustawia `project.in_css_program = true` . Ta flaga identyfikuje projekty do automatycznego downgrade po trialu.

Auto-downgrade — Artisan Command

```
php artisan workspace:downgrade-css-trial
```

Komenda rejestrowana w schedulerze (`app/Console/Kernel.php`) z `->daily()` w bloku `run_landlord_cron` .

Algorytm:

- 1. Pobiera workspace-y z `trial_ends_at < today()` .
- 2. Pomija te z aktywną subskrypcją Stripe (`$workspace->subscribed('subscription')`).
- 3. Dla każdego: znajdź projekty z `in_css_program=true` i `project_type != SIMPLE` .
- 4. Zmień `project_type = SIMPLE` — blokuje funkcje kampanijne, monitoring, output feedy.
- 5. Loguje per-projekt. Błędy per-rekord nie przerywają pętli.

Diagram ścieżki CSS — pełny

